Flask Login System - Project Documentation (Pavan)

1. Introduction

This Flask blueprint-based login system is designed with user registration, login, OTP email verification, and session handling. SQLite is used as the backend for storing user credentials securely with hashed passwords.

2. Packages Used

1. Flask: For building the web application and managing routes.

2. sqlite3: For creating and interacting with the SQLite database.

3. random: To generate OTPs.

4. smtplib & email.mime: To send OTP emails using Gmail's SMTP server.

5. os: To access environment variables for secure credentials.

6. werkzeug.security: For hashing and checking passwords securely.

7. dotenv: To load environment variables from a .env file.

3. SQLite Database Schema

Database: login/users.db

Table: users

Columns:

- id (INTEGER, Primary Key, Auto Increment)

- username (TEXT, Unique, Not Null)

- email (TEXT, Unique, Not Null)

- password (TEXT, Hashed, Not Null)

- verified (INTEGER, Default 0)

- otp (TEXT): Stores the OTP for verification

4. Functionality Breakdown

1. Registration (/register):

   - Accepts username, email, and password.

   - Hashes the password before storing.

   - Sends OTP to email using Gmail's SMTP server.

2. OTP Verification (/verify/<username>):

   - Accepts the OTP entered by user.

   - Verifies and updates the user's verification status.

3. Login (/login):

   - Accepts username and password.

   - Checks hashed password and verifies if the account is verified.

4. Logout (/logout):

   - Clears session and logs out user.

5. Setting up Email for OTP

Flask Login System - Project Documentation (Pavan)

Environment Variables Required:

- EMAIL_USER: Your Gmail address

- EMAIL_PASS: Your Gmail app password

Steps to get Gmail App Password:

1. Enable 2-Step Verification on your Google account.

2. Go to Google Account > Security > App Passwords.

3. Generate an app password for "Mail" and "Other" (like Flask app).

4. Use the generated password as EMAIL_PASS.

6. Environment Setup (.env)

Create a .env file in your project directory with the following content:

EMAIL_USER=your_gmail_address@gmail.com

EMAIL_PASS=your_generated_app_password

Then use `load_dotenv()` from the `dotenv` package to load them securely.

7. Conclusion

This secure login system demonstrates the integration of email-based OTP verification with hashed

credentials. It's a solid foundation for building a user authentication module for larger applications.